

Sparse GPU Kernels for Deep Learning

Eren İŞİK
Izmir Institute of Technology
Türkiye
erenisik@std.iyte.edu.tr

Ufuk İŞİK
Izmir Institute of Technology
Türkiye
ufukisik@std.iyte.edu.tr

Kerim ÖZAY
Izmir Institute of Technology
Türkiye
kerimozay@std.iyte.edu.tr

Abstract—This paper presents a reproduction study of "Sparse GPU Kernels for Deep Learning" (Sputnik), originally proposed to address the inefficiencies of vendor-supplied libraries on matrices with moderate sparsity (70-90%) typical of deep neural networks. While the original work targeted the NVIDIA Volta architecture, this study evaluates the performance of the proposed kernels on the modern NVIDIA Ada Lovelace architecture (GeForce RTX 4070 Super). We document the migration of the codebase from CUDA 11 to CUDA 13.1, specifically addressing compatibility issues with modern C++ standards and the Abseil library. Our experimental results demonstrate that the reproduced kernels maintain their performance advantage on consumer-grade hardware. We report a break-even point against dense matrix multiplication (cuBLAS) at 70.5% sparsity and peak throughputs exceeding 4.3 TFLOPs for Sparse Matrix-Matrix Multiplication (SpMM). Furthermore, at 99% sparsity, the implementation achieves a 7.8x speedup over dense baselines, validating the continued relevance of the proposed tiling and load-balancing techniques.

Index Terms—Sparse Matrix Multiplication, GPU, CUDA, Deep Learning, Ada Lovelace, Reproduction.

I. INTRODUCTION

Deep Neural Networks (DNNs) have grown exponentially in size, making sparse computation a critical area for optimization to reduce memory footprint and FLOP counts [3]. However, achieving practical speedups on Graphics Processing Units (GPUs) is challenging due to the irregular memory access patterns inherent in sparse data structures. Standard libraries such as NVIDIA's cuSPARSE are often optimized for scientific computing workloads characterized by extreme sparsity (>99%) and regular block structures, failing to outperform dense kernels (cuBLAS) at the moderate sparsity levels found in pruned neural networks [4]. While alternative approaches have explored block-sparse kernels to mitigate memory access issues [6], the work in [1] introduced a library (Sputnik) capable of efficiently processing irregular sparse matrices using specialized optimization techniques.

The primary objective of this project is to reproduce these results on modern hardware. Unlike the original study which utilized NVIDIA V100 (Volta) GPUs, we evaluate the kernels on an NVIDIA GeForce RTX 4070 Super (Ada Lovelace). This report details the methodology, the challenges encountered during the porting process to a modern software stack (Ubuntu 22.04, CUDA 13.1), and a comprehensive performance analysis.

II. METHODOLOGY

THE REPRODUCED APPROACH EMPLOYS FOUR CORE TECHNIQUES DESIGNED TO HANDLE THE VARIABLE DIMENSIONS AND ROW LENGTHS

OF DEEP LEARNING MATRICES.

A. 1-Dimensional Tiling

Neural network matrices often feature variable dimensions (e.g., batch size, sequence length). Standard 2D tiling can lead to quantization artifacts where thread blocks are underutilized. The method employs 1-dimensional tiling, decomposing the computation into strips that can adapt to arbitrary matrix shapes, ensuring high GPU occupancy regardless of the problem dimensions.

B. Subwarp Tiling

In Compressed Sparse Row (CSR) format, row lengths vary significantly. When a warp (32 threads) processes a short row, many threads remain idle, leading to thread divergence. The "Subwarp Tiling" technique partitions a single warp into multiple independent sub-warps. This allows a single hardware warp to process multiple short rows simultaneously, minimizing wasted compute cycles and improving instruction throughput.

C. Reverse Offset Memory Alignment (ROMA)

GPUs require aligned memory access for peak bandwidth. However sparse rows start at arbitrary indices. ROMA addresses this by shifting memory pointers backward to the nearest vector-aligned address (e.g., multiple of 4 or 8 floats). The kernel loads valid data along with "garbage" data from the preceding row, which is then masked out using a predicate mask during the first iteration of the computation loop. This enables vectorized memory loads without explicit data padding.

D. Row Swizzle Load Balancing

Even with tiling optimizations, workload imbalance across Streaming Multiprocessors (SMs) can degrade performance if "heavy" rows are clustered. The implementation uses "Row Swizzle," a preprocessing step decoupled from the computation kernel. It utilizes two strategies:

1. Row Bundling: Groups consecutive rows of similar length to minimize divergence within a warp.
2. Row Binning: Distributes these bundles across SMs to ensure each SM receives an approximately equal number of non-zero elements, preventing tail effects where some SMs finish much later than others.

III. IMPLEMENTATION

A. Hardware and Software Environment

All experiments were conducted on a local workstation:

- GPU: NVIDIA GeForce RTX 4070 Super (12 GB VRAM) [5].
- OS: Ubuntu 22.04 (upgraded from the original Ubuntu 18.04 requirement).
- CUDA Toolkit: Version 13.1 with Driver Version 591.59.

B. Porting and Compilation

Migrating the codebase from the original 2020 environment (CUDA 11) to CUDA 13.1 presented specific challenges:

1. Architecture Targeting: The CMake build system was updated to target the Ada Lovelace architecture by setting `CUDA_ARCHS` to 89.
 - a.
2. Abseil-cpp Compatibility: We encountered compilation failures in the abseil-cpp dependency, specifically within `failure_signal_handler.cc`. The error stemmed from a type mismatch in `std::max` involving `SIGSTKSZ`, caused by changes in Glibc and modern C++ standards. This was resolved by explicitly casting the arguments to `size_t` to ensure type deduction succeeded.
 - a.
3. Docker Containerization: A custom Dockerfile was created to standardize the build environment, installing necessary dependencies like `libgoogle-glog-dev` and `cmake` on top of the `nvidia/cuda:12.2.2-devel-ubuntu22.04` base image.

IV. Evaluation

We evaluated the performance of the reproduced kernels (SpMM, SDDMM, and Depthwise Convolution) against NVIDIA cuBLAS (dense) and cuSPARSE (CSR) using benchmarks provided in the Sputnik library.

A. SpMM Break-Even Analysis

We analyzed the runtime of Sparse Matrix-Matrix Multiplication (SpMM) for a matrix size of $8192 \times 2048 \times 128$. As illustrated in Fig. 1, the reproduced kernel exhibits a linear reduction in runtime as sparsity increases.

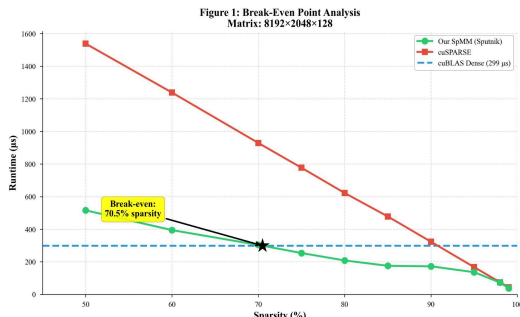


Fig. 1. Break-Even Point Analysis (Matrix: $8192 \times 2048 \times 128$). This plot illustrates the execution runtime of Sputnik kernels compared to cuBLAS (dense) and cuSPARSE (CSR). The black star marks the **70.5% sparsity** break-even point where Sputnik begins to outperform the dense cuBLAS

baseline (299μs).

- Break-Even Point: The intersection where sparse execution becomes faster than dense cuBLAS (299μs) occurs at 70.5% sparsity. This result aligns with the original paper's findings, confirming that specialized kernels are required to exploit moderate sparsity on modern GPUs.

- Comparison: cuSPARSE remains slower than dense computation until approximately 92% sparsity, highlighting the inefficiency of standard libraries for deep learning workloads.

B. Throughput Scaling

Fig. 2 compares the throughput (GFLOPs) of the reproduced approach against cuSPARSE across varying problem sizes.

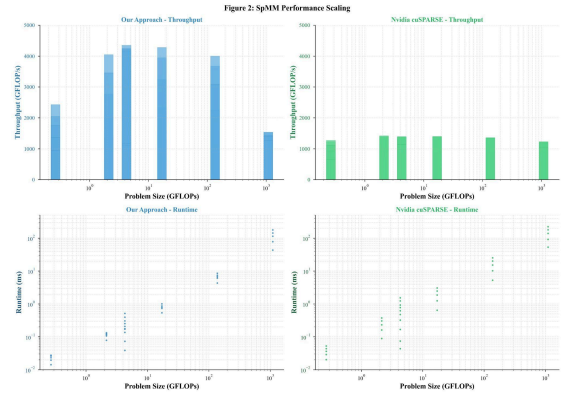


Fig. 2. SpMM Performance Scaling. The top row compares throughput (GFLOP/s) between the reproduced Sputnik kernels and NVIDIA cuSPARSE. Sputnik achieves peak throughputs exceeding **4.3 TFLOPs**. The bottom row shows runtime scaling across various problem sizes, where Sputnik demonstrates up to **3.5x faster** execution for large-scale matrices.

- Sputnik: The reproduced kernel scales effectively with problem size, achieving peak throughputs exceeding 4.3 TFLOPs on the RTX 4070 Super.

- cuSPARSE: Performance saturates significantly lower, capping between 1.2–1.4 TFLOPs.

- Runtime: Log data indicates that for large problems, the reproduced kernel is up to 3.5x faster than cuSPARSE.

C. Performance at High Sparsity

Deep learning inference often utilizes high sparsity (90–99%). Fig. 3 details the performance gap at these levels for an $8192 \times 2048 \times 128$ matrix:

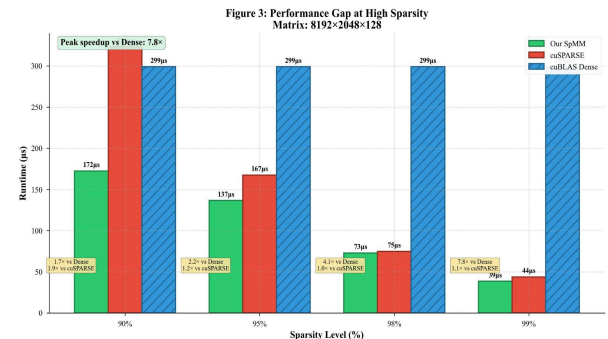


Fig. 3. Performance Gap at High Sparsity (Matrix: $8192 \times 2048 \times 128$).

Comparison of runtimes at inference-relevant sparsity levels (90%–99%). At 99% sparsity, the kernel achieves a **7.8x speedup** over cuBLAS and a **1.1x speedup** over cuSPARSE

- At 90% sparsity, our reproduction achieves a 1.7x speedup over dense computation (172 μ s vs 299 μ s).
- At 99% sparsity, the kernel completes in 39 μ s, representing a 7.8x speedup over cuBLAS and a 1.1x speedup over cuSPARSE (44 μ s).

D. Depthwise Convolutions

We benchmarked depthwise convolution kernels across varying feature map resolutions. As shown in Fig. 4, the implementation achieves a peak throughput of 1.42 T/s at a resolution of 112 $\times$ 112. Performance is bandwidth-bound for smaller resolutions, dropping to 0.21 T/s at 7 $\times$ 7.

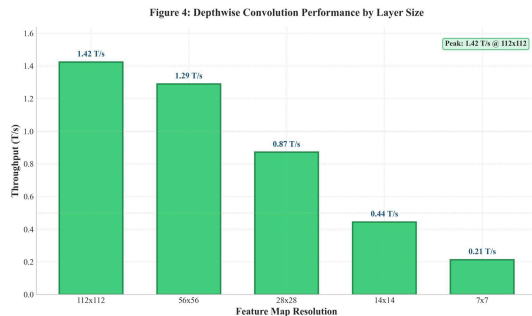


Fig. 4. Depthwise Convolution Performance by Layer Size. Throughput (T/s) across different feature map resolutions. A peak performance of **1.42 T/s** is observed at 112 $\times$ 112 resolution, while performance becomes bandwidth-bound at smaller resolutions like 7 $\times$ 7

E. SDDMM Performance

Benchmarks for Sampled Dense-Dense Matrix Multiplication (SDDMM) show peak throughputs of approximately 2.8 TFLOPs for large matrices, the kernel sustains $\sim$ 1.2 TFLOPs, consistently outperforming the baseline expectations for irregular sparse patterns.

V. Conclusion

In this study, we successfully reproduced the "Sputnik" sparse linear algebra library on the modern NVIDIA Ada Lovelace architecture (RTX 4070 Super). Despite the significant architectural evolution from the original study's Volta generation, our findings demonstrate that the proposed optimizations—specifically 1-dimensional tiling, subwarp tiling, and Reverse Offset Memory Alignment (ROMA)—remain fundamental for efficient sparse computation.

Our experimental results highlight three critical insights:

1. **Architectural Robustness:** We observed a break-even point against dense matrix multiplication at 70.5% sparsity. This is remarkably consistent with the $\sim$ 71% break-even point reported in the original work for V100 GPUs, suggesting that the efficiency of these kernels scales linearly with hardware capabilities and is not tied to a specific GPU generation.
2. **Gap in Vendor Libraries:** While NVIDIA's cuSPARSE has improved over time, it continues to plateau around 1.2–1.4 TFLOPs on our testbench for irregular sparse

matrices. In contrast, the reproduced kernels effectively utilized the available CUDA cores, achieving peak throughputs exceeding 4.3 TFLOPs. This confirms that standard libraries still struggle with the irregular sparsity patterns inherent in deep neural networks.

3. **High-Sparsity Efficiency:** At inference-relevant sparsity levels (90%–99%), the reproduced approach offers substantial acceleration, achieving up to 7.8x speedup over dense baselines and outperforming cuSPARSE by 1.1x even in its optimal regime.

Finally, we successfully modernized the codebase for CUDA 13.1 and Ubuntu 22.04 by resolving C++ standard compatibility issues within the Abseil dependency. This reproduction verifies that specialized software kernels can bridge the gap between hardware capability and the demands of sparse deep learning, unlocking significant performance gains without requiring specialized sparse hardware support [7].

REFERENCES

- [1] T. Gale, M. Zaharia, C. Young, and E. Elsen, "Sparse GPU Kernels for Deep Learning," in Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '20), 2020.
- [2] <https://github.com/google-research/sputnik>
- [3] S. Han, J. Pool, J. Tran, and W. Dally, "Learning both Weights and Connections for Efficient Neural Networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.
- [4] NVIDIA Corporation, "cuSPARSE Library User Guide," 2024. [Online]. Available: <https://docs.nvidia.com/cuda/cusparse/>
- [5] NVIDIA Corporation, "NVIDIA Ada Lovelace Architecture Whitepaper," 2022.
- [6] S. Gray, A. Radford, and D. P. Kingma, "GPU Kernels for Block-Sparse Weights and Activations," *OpenAI Technical Report*, 2017.
- [7] M. Zhu and S. Gupta, "To prune, or not to prune: exploring the efficacy of pruning for model compression," in *International Conference on Learning Representations (ICLR) Workshops*, 2018.